CellHack2

Quick start Rules Reference About

Quick start

You write one function in x86-64 assembly. Every tick, the game calls it once for each cell in your colony, and the cell does whatever the function returns. The colony with the most cells at the end wins.

The game

- `ACT_REST` earns 1 energy.
- `ACT_EAT + dir` takes up to 2 energy from the cell in that direction.
- `ACT_FEED + dir` gives 2 of your energy to the cell in that direction.
- `ACT_SPLIT + dir` creates a new cell in that direction with half your energy. You need at least 5.
- `ACT_MOVE + dir` steps into that square.
- A cell at 0 energy is dead.
- Cells act **fastest thinker first**: the fewer instructions your code ran, the earlier your turn comes. A cell that gets bitten before its turn is **stunned** and loses its action. You have 1000 instructions per call; past that, the cell does nothing this tick.
- After 3000 ticks, most cells wins.

`dir` is `DIR_UP`, `DIR_RIGHT`, `DIR_DOWN` or `DIR_LEFT`. Nothing costs energy except feeding and splitting.

Your function

Register	Holds
`rdi`	pointer to the **view**: your surroundings, your position, a random byte
`rsi`	your energy, 0 to 50
`rdx`	pointer to 64 bytes of **memory** that persist between calls and are copied to your children

Register	Holds
`al`	**your action**, returned with `ret`

Any register may be clobbered. The stack is yours. Every constant used on this page is predefined; there is nothing to include.

Your first cell

```
    mov al, ACT_REST
    ret
```

Two instructions. It rests forever and gets rich. Paste it in as it is.

A cell that grows

```
    ; what is in the square above me?

    lea     r8, [rdi + VIEW_CELLS]
    mov     al, BYTE [r8 + VIEW_UP]
    cmp     al, CELL_EMPTY
    jne     rest

    ; enough energy to split?

    cmp     rsi, 6
    jb      rest
    mov     al, ACT_SPLIT
    add     al, DIR_UP
    ret

rest:
    mov     al, ACT_REST
    ret
```

`VIEW_CELLS` is a 5×5 picture of the squares around you, one byte each, row by row, with you in the middle. Each byte is `CELL_EMPTY`, `CELL_FRIEND`, `CELL_ENEMY` or `CELL_WALL`. `VIEW_UP`, `VIEW_RIGHT`, `VIEW_DOWN` and `VIEW_LEFT` are the indices of the four squares next to you. `byte [rdi + VIEW_RAND]` is a fresh random byte every call. The reference has the rest of the view.

Getting on the ladder

1. **My cells → New cell.** Paste a cell, or start from one of the bots. Say whether it is hand-crafted or written with AI help: it is on your honor, and the leaderboard has a board for each. It assembles immediately and shows you the assembler output.
2. **Sandbox.** Run it against Hunter on a small board and watch the replay.
3. Read the **fault log** on the match page: a crash, a bad action, or more than 1000 instructions in one call shows up there, with the instruction address.
4. Edit, save, repeat. When it is ready, **Put on the ladder**. The ladder runs four-colony matches around the clock and rates each cell on its own. You can have up to three cells on the ladder at once; editing a cell that has played makes a new version, so the old one's record stays.
5. Old replays are deleted after a while. **Save** on a match page keeps that match and its replay for good, up to three per player; the Saved button on the matches page lists yours. **Record video** under the replay saves the match as a video file, with the players named.

The bots

These play on the ladder from day one as 🤖 players. Their code is open: read it, copy it as a starting point, then beat it. A cell that starts from a bot still counts as hand-crafted if no AI touched it.

Bot	What it does	
🤖 Hunter hunter.asm	eats any enemy next to it, otherwise splits when fat, otherwise rests.	Copy as mine
🤖 Idle idle.asm	rests forever. the simplest possible cell.	Copy as mine
🤖 Medic medic.asm	when rich, feeds the friend above it, otherwise splits when fat, otherwise rests.	Copy as mine
🤖 Random random.asm	rests until it has 6 energy, then does something random: a move, split, eat or feed in random direction.	Copy as mine
🤖 Walker walker.asm	splits when it is fat, otherwise randomly decides between moving or resting.	Copy as mine